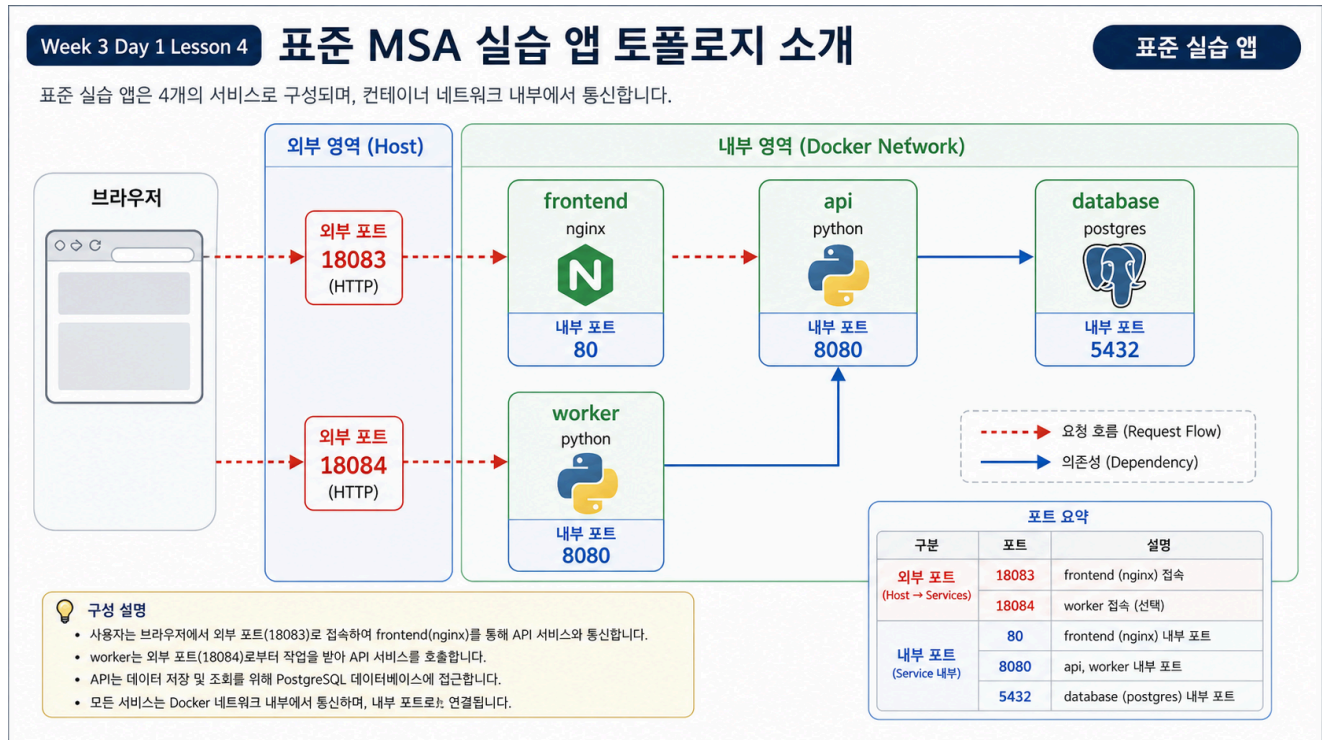


## 4교시: 표준 MSA 실습 앱 토폴로지



### 수업 목표

- msa-demo의 frontend, api, worker, db 요청 흐름을 설명한다.
- 사용자 요청 경로와 background worker 경로를 구분한다.
- topology 그림과 compose.yaml을 서로 연결해서 읽는다.

### 전체 구조

msa-demo는 작은 앱이지만 운영 관점에서 중요한 요소를 모두 가진다.

```
browser
-> frontend nginx
    -> api
        -> db

worker
-> api
-> db
```

서비스별 역할:

| Service  | 역할                             | 사용자 요청 경로 포함 여부 |
|----------|--------------------------------|-----------------|
| frontend | 정적 페이지 제공, /api/ reverse proxy | 포함              |
| api      | 상태 API 제공, DB 연결 확인            | 포함              |
| worker   | 주기적으로 API 상태 확인                | 미포함             |
| db       | PostgreSQL backing service     | 직접 미포함          |

## frontend nginx 설정 읽기

```
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    location /api/ {
        proxy_pass http://api:8080/api/;
        proxy_set_header x-request-id $http_x_request_id;
        proxy_connect_timeout 2s;
        proxy_read_timeout 5s;
    }
}
```

해석:

| 설정                              | 의미                                      |
|---------------------------------|-----------------------------------------|
| listen 80                       | frontend container 내부 port는 80이다.       |
| location /                      | 정적 HTML을 제공한다.                          |
| location /api/                  | browser의 /api/ 요청을 api service로 넘긴다.    |
| proxy_pass http://api:8080/api/ | nginx container가 Compose DNS api를 사용한다. |
| proxy_set_header x-request-id   | request id를 API log로 넘긴다.               |
| proxy_connect_timeout 2s        | API 연결 지연이 무한 대기로 가지 않게 한다.             |

## API 코드에서 보는 dependency

API는 요청이 올 때 DB socket 연결을 시도한다.

```
DB_HOST = os.environ.get("DB_HOST", "db")
DB_PORT = int(os.environ.get("DB_PORT", "5432"))
```

그리고 /health, /api/status에서 DB 연결 가능 여부를 JSON으로 반환한다.

중요한 점:

```
api process가 살아 있음 != api가 service ready 상태임
```

DB 연결이 실패하면 API는 running이어도 /health에서 503을 줄 수 있다.

## worker 경로

worker는 외부 port가 없다.

```
API_URL = os.environ.get("API_URL", "http://api:8080/api/status")
```

worker는 주기적으로 API를 호출하고 결과를 log로 남긴다. 사용자가 worker를 직접 호출하지 않지만, worker 로그는 내부 dependency 상태를 보여주는 좋은 증거가 된다.

## 실습 명령

```
cd week3/day1/labs/msa-demo
docker compose config
```

다음 항목을 topology 그림에 표시한다.

| 항목                                 | 표시 위치               |
|------------------------------------|---------------------|
| 18083:80                           | browser -> frontend |
| proxy_pass http://api:8080/api/    | frontend -> api     |
| DB_HOST=db                         | api -> db           |
| API_URL=http://api:8080/api/status | worker -> api       |
| msa-db-data                        | db -> volume        |

## 잘못 이해하기 쉬운 지점

| 오해                   | 정정                              |
|----------------------|---------------------------------|
| frontend가 DB에 직접 붙는다 | frontend는 API만 호출한다.            |
| worker가 사용자 요청을 처리한다 | worker는 background 확인 루프다.      |
| host에서 DB에 접속해야 한다   | 이 실습에서 DB는 host port를 공개하지 않는다. |
| API가 Up이면 DB도 정상이다   | /health JSON을 봐야 한다.            |

## Evidence Note

```
# W3D1S4 Topology
- user path:
- worker path:
- frontend proxy target:
- api database target:
- stateful volume:
```

## 핵심 포인트

토폴로지는 그림을 예쁘게 그리는 일이 아니다. 요청이 어디로 흐르고, 어느 지점이 실패하면 어떤 service log를 봐야 하는지 결정하는 운영 지도다.